COMPETITIVE PROGRAMMING / ALGORITHM NOTES

Stack / Deque を用いる線形時間処理 48 選

単調スタック・単調キュー・SWAG・ブロック統合・文字列とグラフ

目的　「各長さ K の区間の最小値」のような基本形から、区間包含の除去・Cartesian Tree・凸包・CHT・単調回帰・Tarjan SCC まで、スタックを活用する代表的な処理を系統立ててまとめる。

収録数	48 項目（先の回答の番号を維持。分野をまたぐ重複項目には相互参照を付けた）
基本計算量	$O(N)$ または $O(N+M)$（必要な整列・演算コストは個別に明示）
実装	C++20 の代表テンプレート、式・償却解析・利用条件
対象	競技プログラミングの中級〜上級、紙ライブラリの索引

目次

1 まず覚えたい考え方 3

2 単調スタック：最近傍・寄与計算・幾何（1–15） 3

3 単調 deque：スライド窓・区間 DP（16–23） 5

4 2-stack queue / SWAG：結合的な区間集約（24–30） 7

5 ブロック統合・支配候補の削除・凸包（31–37） 8

6 通常のスタック：構文・文字列・グラフ（38–48） 9

7 一覧・選び方の索引（全 48 項目） 12

8 典型問題・学習順序 13

記法と注意：N は数列・文字列の長さ、M は区間や辺の個数、Q はクエリ数。「線形」は通常、比較やモノイド演算を $O(1)$ とみなすモデルでの計算量。整列が必要な処理は「整列後 $O(N)$」と明記する。また、**単調スタック**と**単なるスタック**、**単調 deque** と **2-stack queue** は別の仕組みである。

1　まず覚えたい考え方

償却 $O(N)$ の原理

配列を左から走査し、候補をスタックへ push する。新しい要素が来たら、将来二度と答えに寄与しない候補を末尾から pop する。

各要素が高々 1 回 push され、高々 1 回 pop される

$$\#\text{push} \le N, \quad \#\text{pop} \le N \quad \Longrightarrow \quad O(N).$$

ループ内部に `while` があっても、毎回 $O(N)$ 回実行されるわけではない。

手法の選び方

問題の特徴	着目する性質	候補
最近傍の大小を知りたい	値で候補を支配し、不要な過去を削除	単調スタック
すべての区間での min/-max	各位置の「最小/最大となる最大範囲」を求める	単調スタック＋寄与
窓の左右端が右に動く	古い候補が期限切れになり、新候補が劣る候補を支配	単調 deque
窓の積・gcd・関数合成	結合律はあるが値の大小関係は使えない	SWAG
隣接ブロックが不適切	末尾ブロックをマージして構造を復元	ブロックスタック
点や直線の包絡線	幾何学的に中間候補が不要になる	Andrew / CHT
入れ子構造・DFS	LIFO 順に処理する必要がある	通常のスタック

よくある誤用：任意順序で与えられる RMQ `min(A[L..R])` を単調スタックだけで $O(N+Q)$ 処理できるわけではない。窓の端点が単調に動くなど、**捨てた候補が復活しない**条件が必要。

2　単調スタック：最近傍・寄与計算・幾何（1–15）

1.　左・右の最近傍の小さい要素　$O(N)$

各 i に対し、左側で最も近い $A_j < A_i$ を求める。`A[st.back()] >= A[i]` の間 pop すれば、残った末尾が答え。右側は逆向き走査で同様。等号の扱いで「厳密に小さい」「以下」を使い分ける。

```
vector<int> prev_smaller(const vector<int>& a) {
    int n = a.size();
    vector<int> L(n, -1), st;
    for (int i = 0; i < n; ++i) {
        while (!st.empty() && a[st.back()] >= a[i])
            st.pop_back();
        if (!st.empty()) L[i] = st.back();
        st.push_back(i);
    }
    return L;
}
```

2.　左・右の最近傍の大きい要素　$O(N)$

1 の不等号を逆転する。左側で最近傍の $A_j > A_i$ なら `A[st.back()] <= A[i]` の間 pop。株価の次の上昇日、右側の次の大きい要素などに使う。

3. Stock Span（連続する過去の非超過範囲） $O(N)$

A_i 以前で $A_j > A_i$ を満たす最も近い j を L_i とすると、i を末尾とする A_i 以下の連続区間長は $i - L_i$。2 とほぼ同じ実装になる。

4. 各要素が区間最小値となる最大範囲 $O(N)$

左側の最も近い 厳密に小さい位置 L_i、右側の最も近い 以下の位置 R_i を取る。A_i が「最小値を取る最も右側の添字」となる区間は $L_i < l \le i \le r < R_i$。等しい値で二重計上しないため、左右で等号を非対称に扱う。

5. 全部分配列の最小値の総和 $O(N)$

4 の範囲を使い、各 i を代表元とする区間の個数が $(i - L_i)(R_i - i)$ であることから

$$\boxed{\sum_{0 \le l \le r < N} \min_{l \le j \le r} A_j = \sum_{i=0}^{N-1} A_i (i - L_i)(R_i - i).}$$

負数があっても成立する。答えが大きくなる場合は `long long` 等を使う。

6. 全部分配列の最大値の総和 $O(N)$

5 の大小を反転する。差を取ることで、全部分配列について $\max - \min$ の総和も $O(N)$ で求められる。

7. ヒストグラム最大長方形 $O(N)$

各 i を長方形の最小高さとみなし、左右の最近傍の厳密に小さい位置を L_i, R_i とする。

$$\boxed{\max_i A_i (R_i - L_i - 1).}$$

同じ高さが連続する場合も、厳密に小さい境界を使えば最大長方形が失われない。各行をヒストグラムにすると 0/1 行列内の最大 1 長方形にも使える。

8. 全ての窓長 K の「区間最小値の最大値」 $O(N)$

全 $K = 1, \dots, N$ について $F(K) = \max_l \min(A_l, \dots, A_{l+K-1})$ を求める。各 i から、左右の厳密に小さい位置による最大幅 $w_i = R_i - L_i - 1$ を計算し、$B[w_i] \leftarrow \max(B[w_i], A_i)$。最後に $K = N-1, \dots, 1$ の順で $B[K] \leftarrow \max(B[K], B[K+1])$ とすれば $F(K) = B[K]$。

9. Cartesian Tree の構築 $O(N)$

二分木の中序走査順が $0, 1, \dots, N-1$、親子の値が min-heap 条件を満たす木。値が小さい要素が現れたときに、大きいスタック末尾をまとめて左部分木に付け直す。最小値 RMQ と LCA の関連や、長方形・分割統治に現れる。重複値では左右どちらを優先するかを固定する。

10. Trapping Rain Water（雨水の総量） $O(N)$

高さの減少が続く間は添字を保持し、新たに右壁が現れたら谷を pop。谷の左右壁の低い方との差に、壁間の幅を掛けて水量を追加する。左右最大値配列を用いる別の $O(N)$ 解法もある。

11. ビルの可視性・見える組の数

$O(N)$（判定条件による）

例えば「間にある全ての高さが両端の低い方よりも低いとき見える」といった遮蔽が高さ比較だけで決まる問題で、スタックに残る未遮蔽の高さを使う。同じ高さのまとめ方や、どちらの壁も数えるかで実装は変わる。任意の幾何的な視線判定まで線形になるわけではない。

12. 132-pattern の存在判定

$O(N)$

$i < j < k$ かつ $A_i < A_k < A_j$ を満たす組があるか判定。右から走査し、値が減少するスタックと「A_k の候補最大値」を管理する。A_i がその候補を下回ったら存在する。反転・大小反転によるパターン判定も同様。

13. 包含関係にある区間の削除（ARC232 A の前処理）

整列後 $O(M)$

左端が厳密に増える順に処理すると、スタック末尾の区間 $[a, b]$ が新しい区間 $[l, r]$ を包含する条件は $b \geq r$。この間 pop し、外側の区間を捨てる。ARC232 A では同じ区間和 2 なので、さらに $[a, l-1]$ と $[r+1, b]$ の要素を 0 に確定させる。区間差分（いもす法）に記録すれば総時間 $O(N+M)$。**単に pop するだけでは元の制約と同値でなくなる**点に注意。

14. 2 次元 Pareto frontier の更新

$O(N)$（整列後）

(x, y) をともに最大化する場合、x の昇順（同じ x は集約）に点を追加する。新しい点の y が末尾の y 以上なら、新しい点が末尾を支配するので pop。残った点の y は厳密に減少し、非支配点が得られる。整列自体は一般に $O(N \log N)$。

15. 凸包（Andrew の単調チェーン法）

$O(N)$（点の整列後）

点を x, y の辞書順に整列し、下側凸包・上側凸包をそれぞれスタックで構築する。末尾 3 点の外積が非正（反時計回り凸包の場合）なら中点を pop。各点を高々 2 回追加・削除する。全体は通常、整列が支配して $O(N \log N)$。

3 単調 deque：スライド窓・区間 DP（16–23）

単調スタックの末尾からの支配候補の削除に、先頭からの期限切れ候補の削除を加える。窓の端点が単調に右へ動くときに有効である。

16. 長さ K の全ての区間の最小値

$O(N)$

$B_i = \min\{A_i, \ldots, A_{i+K-1}\}$ を列挙する。deque には添字を増加順で入れ、対応値も非減少になるよう末尾を削る。先頭にある有効期限切れ添字 $\leq i - K$ を pop front。先頭が現在の最小値になる。

```
vector<int> sliding_min(const vector<int>& a, int k) {
    deque<int> dq;
    vector<int> ans;
    for (int i = 0; i < (int)a.size(); ++i) {
        while (!dq.empty() && dq.front() <= i-k)
            dq.pop_front();
        while (!dq.empty() && a[dq.back()] >= a[i])
            dq.pop_back();
        dq.push_back(i);
        if (i >= k-1) ans.push_back(a[dq.front()]);
    }
    return ans;
}
```

削除できる理由 $j < i$ かつ $A_j \geq A_i$ なら、今後の窓が j を含む間は i も含む。したがって j は二度と最小値になれない（同値の場合は後から来た i を残す）。

17. 長さ K の全ての区間の最大値 $O(N)$

16 の不等号を逆転する。deque 内で A を非増加に保ち、先頭が最大値となる。

18. 各窓の最大値と最小値の差 $O(N)$

最小値用と最大値用の deque を 2 本並行して維持し、各位置で $\max - \min$ を計算する。固定長窓なら 16 と 17 をそのまま併用する。

19. $\max - \min \leq D$ を満たす最長部分配列 $O(N)$

右端 r を増やし、最小値・最大値の deque を更新する。$\max - \min > D$ の間、左端 l を増やし、期限切れ要素を除く。条件を満たす窓の最大長 $r - l + 1$ を記録する。単調な条件ゆえに尺取りが成立。

20. $\max - \min \leq D$ を満たす部分配列の個数 $O(N)$

19 と同じ l を求めると、右端が r の適合区間は $[l, r], \ldots, [r, r]$ の $r - l + 1$ 個。各 r でその値を加算すればよい。

```
long long count_bounded_range(const vector<int>& a,
                              long long d) {
    deque<int> mn, mx;
    long long ans = 0;
    int l = 0;
    for (int r = 0; r < (int)a.size(); ++r) {
        while (!mn.empty() && a[mn.back()] >= a[r]) mn.pop_back();
        while (!mx.empty() && a[mx.back()] <= a[r]) mx.pop_back();
        mn.push_back(r); mx.push_back(r);
        while ((long long)a[mx.front()] - a[mn.front()] > d) {
            if (mn.front() == l) mn.pop_front();
            if (mx.front() == l) mx.pop_front();
            ++l;
        }
        ans += r-l+1;
    }
    return ans;
}
```

前提：$d \geq 0$。a が int なら差分の計算時に long long へ拡張する。

21. 幅 K の区間 min による DP 最適化 $O(N)$

$$dp[i] = c_i + \min_{\max(0,i-K) \leq j < i} dp[j].$$

既に計算した $dp[j]$ を単調 deque で持ち、各 i で窓から外れた候補を削除してから最小値を読んで $dp[i]$ を計算し、最後に i を追加する。素朴 $O(NK)$ が $O(N)$ になる。

22. 両端が単調に動く区間の min/max クエリ $O(N + Q)$

クエリ区間 $[L_q, R_q]$ の左右端がともに単調非減少であるとき、右端の増加分だけ候補を追加し、左端より古い候補を削除する。各添字の push/pop は高々 1 回。**右端の追加を処理した後に答える**。一般の任意順 RMQ には適用できない。

23. 単調な有効期限をもつ候補 DP

$O(N)$（条件付き）

候補の評価値が固定（または相対順位が後で逆転しない）で、到着順と期限切れ順が一致する場合、優劣で末尾削除・期限切れで先頭削除を行う。**順位の逆転や有効期限の非単調性があると一般には使えない**。優先度付きキュー・セグメント木・CHT との差を確認する。

4 2-stack queue / SWAG：結合的な区間集約（24–30）

単調 deque が使えるのは通常、大小の比較により候補を永久に捨てられる場合。SWAG は結合律だけでキュー全体の集約を維持する。

> 二項演算 $\circ$ が $(a \circ b) \circ c = a \circ (b \circ c)$ を満たせば、**交換律は不要**。2 本のスタック `in/out` に、各スタックの集約値を持たせる。`out` が空のときだけ `in` 全部を移し替えるので、各要素の移動は高々 1 回。

24. Minimum Stack（stack 全体の min/max）

各操作 $O(1)$

各 push 時に、値 x と「ここまでの最小値」$m' = \min(x, m)$ を一緒に入れる。pop で保存した集約値も戻るため、min/max が $O(1)$ で取得できる。

25. Minimum Queue（queue 全体の min/max）

償却 $O(1)$ / 操作

24 の MinStack を 2 本用意。`push_back` は `in`、`pop_front` は `out` を使う。`out` が空なら `in` を全部移動。答えは $\min(\min(\text{out}), \min(\text{in}))$。

26. Sliding Window GCD

総 $O(N)$ 回の gcd 演算

演算を $\circ = \gcd$ とすればよい。整数 1 回の gcd を $O(\log V)$ と見積もるなら全体 $O(N \log V)$。値に単調性がないので、16 の単調 deque をそのまま使うことはできない。

27. Sliding Window Bitwise AND / OR

総 $O(N)$ 回のビット演算

$\circ = \&$ または $\circ = |$ とする。固定ビット幅での演算が $O(1)$ なら全体 $O(N)$。単調スタックは不要で、一般の SWAG の一例。

28. Sliding Window Matrix Product

固定次元なら $O(N)$

行列の積は結合的だが非可換。$A_l A_{l+1} \cdots A_r$ を**順序を崩さず**集約する。行列次元を d とすると通常 $O(Nd^3)$ の算術計算量。

29. Sliding Window Function Composition

関数表示の演算コスト $\times O(N)$

一次関数 $f_i(x) = a_i x + b_i$ の合成を管理する。合成順に注意し、たとえば $f_r \circ \cdots \circ f_l$ を求めるモノイド演算を定義する。係数ペア (a, b) で表せ、1 回の演算は $O(1)$。

30. 汎用 SWAG（Sliding Window Aggregation）

償却 $O(1)$ / 操作

任意のモノイド（結合的演算と単位元）に適用する。以下は `op(a,b)` が順序付きの集約 $a \circ b$ を返す実装。サイズ固定の窓は「push → 必要に応じて pop」を反復する。

```cpp
template<class T, class Op>
struct SWAG {
    vector<pair<T,T>> in, out; // {value, aggregate}
    T id;
    Op op;
    SWAG(T identity, Op operation) : id(identity), op(operation) {}

    void push(T x) {
        T v = in.empty() ? x : op(in.back().second, x);
        in.push_back({x, v});
    }
    void transfer() {
        while (!in.empty()) {
            T x = in.back().first;
            in.pop_back();
            T v = out.empty() ? x : op(x, out.back().second);
            out.push_back({x, v});
        }
    }
    void pop() { // call only when queue is nonempty
        if (out.empty()) transfer();
        out.pop_back();
    }
    T fold() const {
        if (out.empty() && in.empty()) return id;
        if (out.empty()) return in.back().second;
        if (in.empty()) return out.back().second;
        return op(out.back().second, in.back().second);
    }
};
```

注意：`out` への移動時だけ `op(x, aggregate)` と順序が逆になる。これにより非可換演算にも対応する。

5　ブロック統合・支配候補の削除・凸包（31–37）

ここでは「単調な値のスタック」から一歩進め、区間・点・ブロック・直線を候補として積む手法をまとめる。一部は 1 節の再掲だが、適用の切り口が異なる。

31.　区間包含の削除（再掲：13）

整列後 $O(M)$

左端の昇順に処理して、末尾の右端が新しい右端以上なら pop。ARC232 A のように区間和が一致するなら、外側と内側の差集合はすべて 0 と確定する。制約の削除とともにこの情報も記録する。

32.　区間のマージ（区間の和集合）

整列後 $O(M)$

左端昇順にソートして順に見る。新しい $[l, r]$ と末尾 $[a, b]$ が重なれば、$[a, \max(b, r)]$ に統合する。実際にはスタックではなく最後の *1* 区間だけで足りることも多い。整数閉区間で隣接も統合したい場合は $l \le b + 1$ を使う。

33.　オンライン Pareto frontier（再掲：14）

$O(N)$（入力が単調順の場合）

新しい点が過去の点を支配したら pop し、支配されない候補を順序付きスタックに保持する。各点を高々一度しか pop しない。複数座標の任意の支配関係を必ずスタック処理できるわけではない。

34.　Isotonic Regression / PAVA（単調回帰）

$O(N)$

A_i に対して $x_1 \le \cdots \le x_N$ の下で $\sum_i (x_i - A_i)^2$ を最小化する。各位置を初期ブロックにして、平均値が単調性を破る隣接ブロックをまとめる（Pool Adjacent Violators Algorithm）。ブロックは個数と合計を保持す

ればよく、各マージでブロック数が 1 減る。

$$A = [1, 5, 3, 4, 8] \quad \Longrightarrow \quad x = [1, 4, 4, 4, 8].$$

重み付き二乗誤差でも、平均を重み付き平均に置き換えれば同様。

```
// Squared-error isotonic regression (nondecreasing).
struct Block { long double sum; int cnt; };
vector<long double> isotonic(const vector<long double>& a) {
    vector<Block> st;
    for (auto x : a) {
        st.push_back({x, 1});
        while (st.size() >= 2) {
            auto b = st.back(), c = st[st.size()-2];
            if (c.sum / c.cnt <= b.sum / b.cnt) break;
            st.pop_back(); st.pop_back();
            st.push_back({c.sum+b.sum, c.cnt+b.cnt});
        }
    }
    vector<long double> ans;
    for (auto b : st)
        for (int j = 0; j < b.cnt; ++j)
            ans.push_back(b.sum / b.cnt);
    return ans;
}
```

35. Andrew の凸包（再掲：15）

整列後 $O(N)$

3 点の外積により中間点が凸包の境界に必要か判定し、不要なら pop。幾何的な「候補の永久削除」の典型例。全点が共線の場合の頂点の残し方は `cross <= 0` と `cross < 0` で変わる。

36. Convex Hull Trick（CHT）

構築 $O(N)$、単調クエリ総 $O(Q)$

最小値 $\min_j(a_j x + b_j)$ を求めるとき、傾き a_j が厳密に単調であれば、交点が前後関係を乱す中間の直線を pop して下包絡線を構築できる。クエリの x も単調なら、最適な直線が一方向に動くので deque で答えられる。傾きの向き・min/max・クエリ順の組合せに応じて向きを設定する。

37. 凸性を利用する DP 最適化

条件次第で $O(N)$

$dp[i] = \min_{j<i}\{a_j x_i + b_j\}$ のような遷移を直線の最小値クエリに変換する。傾き a_j と x_i が適切な順に単調なら 36 により全体 $O(N)$。単調性が崩れた場合は Li Chao Tree や別のデータ構造を検討する。

6 通常のスタック：構文・文字列・グラフ（38–48）

以下は必ずしも単調スタックではないが、最後に追加した未処理要素から処理する（*LIFO*）ことで、線形時間の処理を実現する典型例である。

38. 括弧列の対応付け

$O(N)$

左括弧の添字を push、右括弧に出会ったら対応する左括弧の添字を pop。対応関係が入れ子になるためスタックで正しく処理できる。括弧の種類が複数ある場合は種類の一致も確認する。

39. 最長の正しい括弧部分文字列

$O(N)$

`'('` の添字を push し、`')'` に出会ったら pop。不一致が起きたら境界を更新する。番兵 -1 をスタックに入れておく実装では、pop 後に空なら現在位置を新しい境界として push、そうでなければ $i -$ top が末尾 i の最長長さになる。

```cpp
int longest_valid_parentheses(const string& s) {
    vector<int> st = {-1};
    int ans = 0;
    for (int i = 0; i < (int)s.size(); ++i) {
        if (s[i] == '(') st.push_back(i);
        else {
            st.pop_back();
            if (st.empty()) st.push_back(i);
            else ans = max(ans, i - st.back());
        }
    }
    return ans;
}
```

入力は'(' と')' のみとする。空スタックへの pop は発生しない（初期番兵を入れ、空になれば直ちに境界を復元）。

40. 隣接要素の消去・文字列簡約 $O(N)$

例えば等しい隣接 2 文字を繰り返し消すなら、次の文字がスタック末尾と等しい場合 pop、異なれば push。消去後に新しく隣り合った文字も処理できる。消去規則に順序依存性がある場合は問題の定義を確認する。

41. 長さ K の辞書順最小部分列 $O(N)$

左から文字を処理し、新しい文字が末尾より小さく、かつ削除しても最終長 K が作れる間は pop する。末尾へ追加するのは現在の長さが K 未満のとき。例えば数列でも辞書順最小の長さ K の部分列を求められる。

```cpp
string smallest_subsequence(const string& s, int k) {
    string st;
    int n = s.size();
    for (int i = 0; i < n; ++i) {
        while (!st.empty() && st.back() > s[i] &&
               (int)st.size()-1 + n-i >= k)
            st.pop_back();
        if ((int)st.size() < k) st.push_back(s[i]);
    }
    return st;
}
```

前提：$0 \le K \le N$。同じ値が複数あっても、辞書順最小の値列を得る。

42. Remove K Digits（K 桁削除して最小値） $O(N)$

41 と同様、より小さい桁が後から来たら大きい末尾を削除する。削除可能回数を持ち、最後に余っていれば末尾から削除。先頭の 0 を取り除くなど、数としての表現を別途整える。

43. 逆ポーランド記法（RPN）の評価 $O(N)$

数値トークンを push、演算子が現れたら引数を pop して計算し、結果を push。演算子の引数順が重要で、減算では `b=pop(); a=pop(); push(a-b)` とする。

44. 中置記法から後置記法への変換 $O(N)$

Shunting-yard 法。演算子の優先度・結合規則を見て、スタック末尾の演算子を出力へ移す。括弧は対応が取れるまで処理する。トークン数に比例した時間で構文を処理できる。

45. 非再帰 DFS

$O(N+M)$

再帰関数の代わりに明示的なスタックを使う。頂点だけを積むと隣接辺の探索順序が再帰と変わる場合がある。完全に再帰を模倣したいときは「頂点と次に調べる隣接辺の番号」をスタックに保持する。

46. Tarjan の強連結成分分解（SCC）

$O(N+M)$

DFS の発見順 `ord`、到達可能な祖先の最小発見順 `low`、未確定の頂点を保持するスタックを使う。$\mathrm{ord}[v] = \mathrm{low}[v]$ を満たす根で、スタックから強連結成分をまとめて pop。*DFS* の再帰スタックと *SCC* 候補スタックは概念上異なる。

47. 木の Euler Tour（入退場時刻）

$O(N)$

DFS において各頂点の入場時に `tin[v]`、退場時に `tout[v]` を記録する。再帰の代わりに入場・退場イベントをスタックへ積めばよい。部分木は Euler Tour 順で連続区間となるため、部分木クエリへ応用できる。

48. 木の部分木 DP の反復実装

$O(N)$

根から DFS 順 `order` を作り、逆順に処理して子から親へ DP を集約する。一般の木では親配列を作り、親への逆流を除いて子だけを足す。再帰深さ $O(N)$ を避けられる。

```cpp
// Subtree sizes, tree adjacency list g, root = 0.
vector<int> par(n, -1), ord = {0};
par[0] = 0;
for (int q = 0; q < (int)ord.size(); ++q) {
    int v = ord[q];
    for (int u : g[v]) if (u != par[v]) {
        par[u] = v;
        ord.push_back(u);
    }
}
vector<int> sz(n, 1);
for (int i = n-1; i > 0; --i) {
    int v = ord[i];
    sz[par[v]] += sz[v];
}
```

木を前提にした実装。`ord` はキュー状の走査でも求められ、順序を逆にすることが本質である。

7　一覧・選び方の索引（全 48 項目）

No.	処理	鍵となる不変条件・機構
1	最近傍の小さい要素	添字増加・値増加のスタック
2	最近傍の大きい要素	添字増加・値減少のスタック
3	Stock Span	最も近い厳密に大きい壁
4	最小値としての最大範囲	左右の非対称タイブレーク
5	全区間最小値の総和	値の寄与 $(i-L)(R-i)$
6	全区間最大値の総和	5 の大小反転
7	ヒストグラム最大長方形	高さ $\times$ 支配されない幅
8	全窓長の「最小値の最大」	最大幅から窓長への降順累積最大
9	Cartesian Tree	新最小値で部分木を付け替え
10	雨水貯留	谷の左右壁が揃ったら確定
11	ビルの可視性	高さによる遮蔽・重複の管理
12	132-pattern	右から走査し中央値候補を更新
13	区間包含の削除	外側区間を pop（確定情報を保存）
14	2 次元 Pareto frontier	新点による過去点の支配
15	Andrew 凸包	外積の符号で中間点を削除
16	各 K 窓の最小値	末尾の劣位候補・先頭の期限切れ
17	各 K 窓の最大値	16 の大小反転
18	窓内の max − min	min/max deque の並列使用
19	変動幅制約の最長部分配列	2 deque ＋尺取り
20	変動幅制約の区間個数	右端固定で $(r-l+1)$ を加算
21	区間最小値 DP	直近 K 個の DP 候補を維持
22	単調端点の RMQ	両端が右にしか動かない
23	単調な期限付き候補 DP	評価順と期限切れ順が一定
24	Minimum Stack	要素と集約値を積む
25	Minimum Queue	2 本の MinStack を使う
26	Sliding Window GCD	2-stack の結合的集約
27	Sliding Window AND / OR	2-stack の結合的集約
28	Sliding Window Matrix Product	非可換でも順序を保持する
29	Sliding Window Function Composition	非可換な合成の順序を保持する
30	汎用 SWAG	モノイド・償却 $O(1)$
31	区間包含削除（13 と同じ）	制約を捨てるとき確定情報を記録
32	区間のマージ	整列済みの区間を統合
33	オンライン Pareto（14 と同じ）	支配された末尾を削除
34	PAVA（単調回帰）	平均値が逆転したブロックをマージ
35	Andrew 凸包（15 と同じ）	外積で不要な点を削除
36	Convex Hull Trick	交点順で不要な直線を削除
37	凸性に基づく DP 最適化	36 に DP を帰着する
38	括弧列の対応付け	開き括弧を後入れ先出し
39	最長正しい括弧部分文字列	スタック＋不一致の直近位置
40	隣接消去・文字列簡約	最後に残った文字と衝突判定
41	長さ K の辞書順最小部分列	長さ制約下で大きい末尾を pop
42	Remove K Digits	桁数制約下で大きい末尾を pop
43	逆ポーランド記法の評価	項を積んで演算子で集約
44	中置 → 後置記法	演算子の優先順位と括弧を管理
45	非再帰 DFS	明示的な DFS スタック
46	Tarjan SCC	発見順・lowlink・候補スタック
47	木の Euler Tour	入場・退場イベントを管理
48	木の部分木 DP	走査順の逆順で子から親へ集約

8　典型問題・学習順序

実際の問題	対応技法
CSES – Nearest Smaller Values	1：単調スタックの基本
CSES – Sliding Window Minimum	16：単調 deque
CSES – Advertisement	7：ヒストグラム
AtCoder ABC189 C – Mandarin Orange	7：区間最小値と区間長
AtCoder ABC372 D – Buildings	11：ビルの可視性とスタック
AtCoder ABC359 E – Water Tank	4・5：単調スタックによる寄与管理
Library Checker – Queue Operate All Composite	29・30：非可換 SWAG
AtCoder ARC232 A – Two in the Range	13・31：区間包含の前処理

おすすめの学習順　最近傍大小 → 最小値の寄与 → ヒストグラム → 単調 deque → 2-stack queue / SWAG → Cartesian Tree → PAVA・凸包・CHT。

処理が線形になるかを判断する 4 つの質問

1. **何を捨てるか？**　捨てる対象が後続の入力で復活しないと証明できるか。
2. **どちらから捨てるか？**　末尾だけなら stack、先頭にも有効期限があるなら deque。
3. **何回捨てるか？**　各要素の push/pop が定数回で済むか。
4. **前処理は？**　ソートが必要か、単調性が最初から保証されているか。

本資料について　前の回答で列挙した 48 項目を 1 つの資料に統合したもの。No.13/31、14/33、15/35 は意図的に番号を保持した重複項目。計算量は「適用条件を満たす場合」の評価であり、汎用のデータ構造が常に不要になることを意味しない。